

Product and Category Management System

Database Design & Implementation Document

This document presents the database design for a Product and Category Management System, covering table structures, key relationships, core CRUD operations, and category-wise reporting queries.

Scope	Design tables, define relationships, manage products, generate reports
Entities	Category, Product
Database	Relational (MySQL / PostgreSQL / SQL Server compatible)
Key Operations	Insert, Update, Delete, Category-wise Reporting

1. Table Design

The system is built around two core entities: **Category** and **Product**. Each category can contain many products, forming a one-to-many relationship.

1.1 Category Table

Column	Data Type	Constraint	Description
category_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for a category
category_name	VARCHAR(100)	NOT NULL, UNIQUE	Name of the category (e.g. Electronics)
description	VARCHAR(255)	NULL	Short description of the category
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time

1.2 Product Table

Column	Data Type	Constraint	Description
product_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for a product
product_name	VARCHAR(150)	NOT NULL	Name of the product
category_id	INT	FOREIGN KEY REFERENCES Category (category_id)	Links product to its category
price	DECIMAL(10,2)	NOT NULL, CHECK (price >= 0)	Selling price of the product
stock	INT	NOT NULL, DEFAULT 0	Available quantity in stock
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation time
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Last update time

2. Primary and Foreign Key Relationships

category_id is the primary key of the Category table and is referenced as a foreign key in the Product table. This establishes a **one-to-many relationship**: one category can have many products, but each product belongs to exactly one category.

Relationship	Category (1) → Product (Many)
Primary Key	Category.category_id
Foreign Key	Product.category_id → Category.category_id
On Delete	RESTRICT / CASCADE (based on business rule)

2.1 DDL Statements

```
CREATE TABLE Category (  
  category_id INT PRIMARY KEY AUTO_INCREMENT,  
  category_name VARCHAR(100) NOT NULL UNIQUE,
```

```
        description    VARCHAR(255),
        created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

CREATE TABLE Product (
    product_id         INT PRIMARY KEY AUTO_INCREMENT,
    product_name       VARCHAR(150) NOT NULL,
    category_id        INT NOT NULL,
    price              DECIMAL(10,2) NOT NULL CHECK (price >= 0),
    stock              INT NOT NULL DEFAULT 0,
    created_at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at         TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                      ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_product_category
        FOREIGN KEY (category_id)
        REFERENCES Category(category_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```

3. Storing Product Details

Every product record stores its name, associated category, price, and available stock, along with timestamps for auditing. Sample data illustrates how records are stored:

product_id	product_name	category_id	price	stock
1	Wireless Mouse	2 (Electronics)	699.00	150
2	Office Chair	3 (Furniture)	4999.00	35
3	Bluetooth Speaker	2 (Electronics)	1899.00	80
4	Notebook Set	1 (Stationery)	120.00	500

4. Product Insertion, Updating, and Deletion

4.1 Insert a Product

```
INSERT INTO Product (product_name, category_id, price, stock)
VALUES ('Wireless Mouse', 2, 699.00, 150);
```

4.2 Update a Product

```
-- Update price and stock after a restock
UPDATE Product
SET price = 749.00,
    stock = stock + 50
WHERE product_id = 1;
```

4.3 Delete a Product

```
DELETE FROM Product
WHERE product_id = 4;
```

Note: Because `category_id` is a foreign key, deleting a Category that still has linked products will fail under the `RESTRICT` rule unless those products are reassigned or removed first.

5. Category-wise Product Reports

A category-wise report joins the Product and Category tables to summarize product count, total stock, and inventory value per category.

```
SELECT
    c.category_name,
    COUNT(p.product_id) AS total_products,
    SUM(p.stock) AS total_stock,
    ROUND(SUM(p.price * p.stock), 2) AS inventory_value
FROM Category c
LEFT JOIN Product p
    ON c.category_id = p.category_id
GROUP BY c.category_id, c.category_name
ORDER BY inventory_value DESC;
```

5.1 Sample Report Output

category_name	total_products	total_stock	inventory_value
Furniture	1	35	174,965.00
Electronics	2	230	256,240.00
Stationery	1	500	60,000.00

6. Summary

- Category and Product tables designed with clear column definitions and constraints.
- One-to-many relationship enforced via category_id as primary key / foreign key.
- Core CRUD operations (INSERT, UPDATE, DELETE) defined for product management.
- Category-wise reporting query aggregates product count, stock, and inventory value.